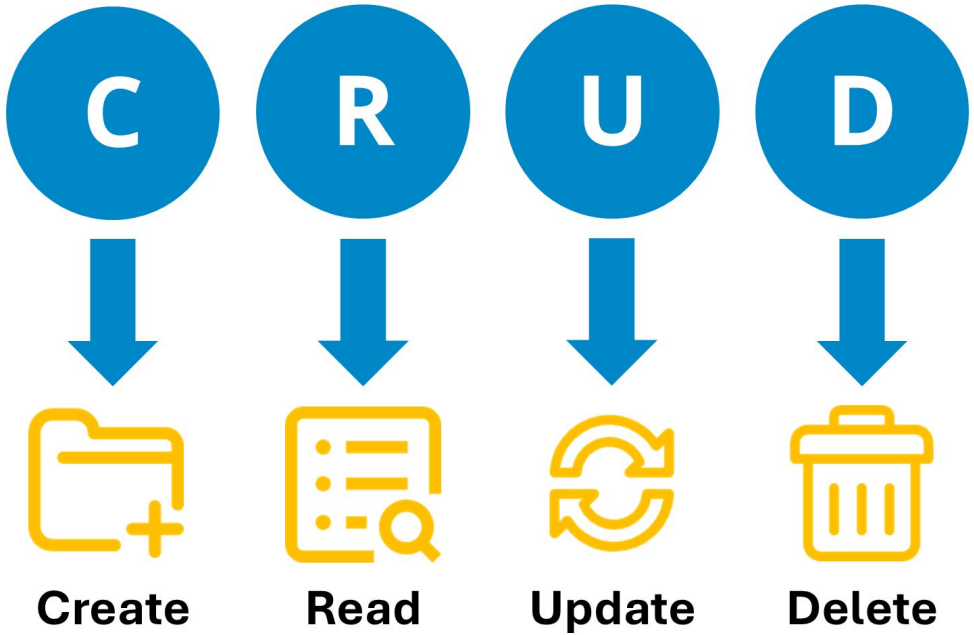
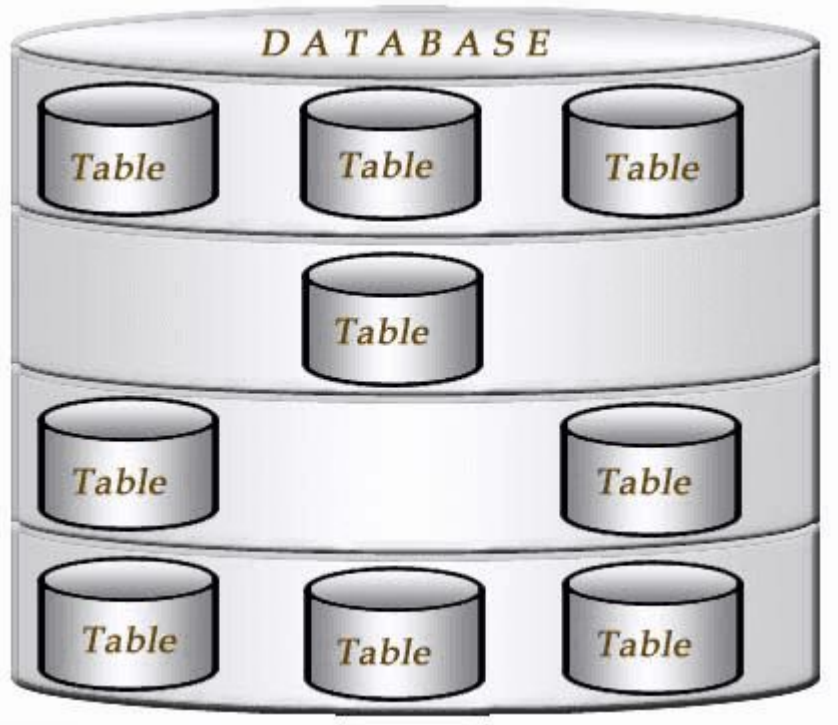
Lesson 1: SQL Basics

Prepared By: Ved Vyas

A **database** is an organized collection of data that is stored and managed so that information can be easily **accessed, added, updated, and deleted**.




DATABASE
Table
Table
Table
Table
Table
Table
Table
Table
Table

A **table** is a structure inside a database that stores related data in **rows and columns**.

Student_ID	Name	Age	Course
101	Rahul	20	CSE
102	Priya	19	CSE
103	Amit	21	IT

- **Column:** Represents an attribute, such as `Name` or `Age`.
- **Row:** Represents one complete record, such as Rahul's information.

Database = Collection of organized data
Table = Organized rows and columns used to store that data

Category	Full Form	Main Commands	Main Purpose
DDL	Data Definition Language	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	Defines database structure
DML	Data Manipulation Language	`INSERT`, `UPDATE`, `DELETE`	Modifies data
DQL	Data Query Language	`SELECT`	Retrieves data
DCL	Data Control Language	`GRANT`, `REVOKE`	Controls permissions
TCL	Transaction Control Language	`COMMIT`, `ROLLBACK`, `SAVEPOINT`	Manages transactions

SQL (Structured Query Language) is a standard language used to **create, store, retrieve, modify, and manage data in relational databases**.

SQL is used with database systems such as **MySQL, PostgreSQL, Oracle, SQL Server, and SQLite**.

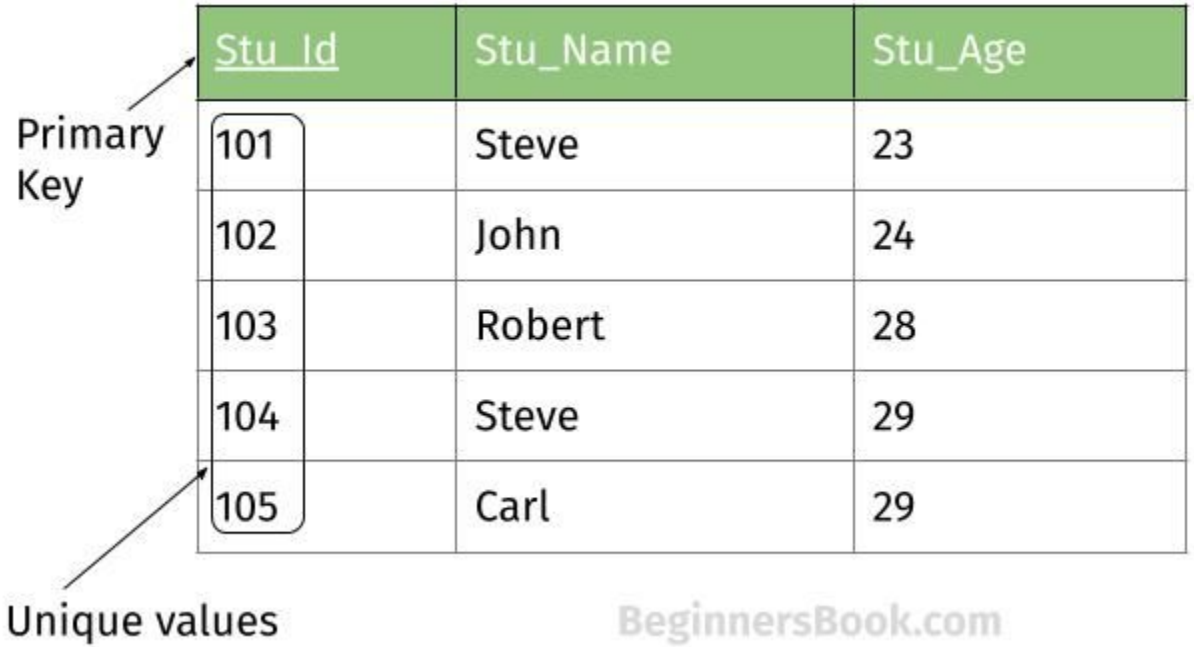
Category	Main Data Types
Numeric	`INT`, `DECIMAL`, `FLOAT`
Character	`CHAR`, `VARCHAR`, `TEXT`
Date & Time	`DATE`, `TIME`, `DATETIME`, `TIMESTAMP`
Boolean	`BOOLEAN`
Binary	`BINARY`, `VARBINARY`, `BLOB`
Special	`JSON`, `ENUM`, `XML`

A **data type** defines the **type of value that can be stored in a variable, field, or database column**. It tells the database what kind of data a column can contain, such as numbers, text, dates, or true/false values.

The `CREATE TABLE` command is used to **create a new table in a database**.

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

`PRIMARY KEY` ensures that every student has a **unique identifier**.



```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Marks DECIMAL(5,2)
);
```

`Student` is the **table name**.

`Student_ID`, `Name`, `Age`, and `Marks` are **columns**.

`INT`, `VARCHAR(50)`, and `DECIMAL(5,2)` are **data types**.

`INSERT` Command

The `INSERT` command is used to **add new records (rows) into a table.**

Basic Syntax

SQL

```sql
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

```
-- Insert first record
INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (101, 'Rahul', 20, 85.50);

-- Insert second record
INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (102, 'Priya', 19, 90.00);

-- Insert third record
INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (103, 'Amit', 21, 78.50);
```

```
INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES
    (101, 'Rahul', 20, 85.50),
    (102, 'Priya', 19, 90.00),
    (103, 'Amit', 21, 78.50);
```

```
7  CREATE TABLE groceries (id INTEGER PRIMARY KEY, name
   TEXT, quantity INTEGER );
8
9  INSERT INTO groceries VALUES (1, "Bananas", 4);
10 INSERT INTO groceries VALUES (2, "Peanut Butter", 1);
11 INSERT INTO groceries VALUES (3, "Dark chocolate bars"
   , 2);
12 SELECT * FROM groceries;
```

`SELECT` Statement

The `SELECT` **statement** is used to **retrieve or display data from a table** in a database.

1. Select All Columns

SQL

```sql
SELECT * FROM Student;
```

Here, `*` means **all columns.**

2. Select Specific Columns

SQL

```sql
SELECT Name, Age, Marks
FROM Student;
```

This displays only the `Name`, `Age`, and `Marks` columns.

3. Select a Specific Record Using `WHERE`

SQL

```sql
SELECT *
FROM Student
WHERE Student_ID = 101;
```

This displays the record where the `Student_ID` is `101`.

Basic Syntax

SQL

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

In simple words:

> `SELECT` is used to **fetch data from a database table.**

SQL

```sql
-- your code goes here
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Marks DECIMAL(5,2)
);

INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (101, 'Rahul', 20, 85.50);

INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (102, 'Priya', 19, 90.00);

INSERT INTO Student (Student_ID, Name, Age, Marks)
VALUES (103, 'Amit', 21, 78.50);

SELECT * FROM Student;
```

Your Output

```
101|Rahul|20|85.5
102|Priya|19|90
103|Amit|21|78.5
```

```sql
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 4, 7);
INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2);
INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT * FROM groceries;
```

id (PK)	INTEGER
name	TEXT
quantity	INTEGER
aisle	INTEGER

QUERY RESULTS

id	name	quantity	aisle
1	Bananas	4	7
2	Peanut Butter	1	2
3	Dark Chocolate Bars	2	2
4	Ice cream	1	12
5	Cherries	6	2
6	Chocolate syrup	1	4

In **DB (Database)**, **schema** means the **structure or blueprint of the database.**

It defines things like:

- What **tables** exist.
- What **columns/fields** each table has.
- What **data types** those columns use.
- What **relationships** exist between tables.
- What **constraints** are applied, such as `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, and `UNIQUE`.

`ORDER BY` is used to **sort the records returned by a query** based on one or more columns.

Sorting means **arranging data in a particular order**. (ascending or descending)

Basic syntax

SQL

```sql
SELECT column1, column2
FROM table_name
ORDER BY column1 ASC;
```

There are two main sorting orders:

- `ASC` → Ascending order (small → large, A → Z). This is the default.
- `DESC` → Descending order (large → small, Z → A).

Suppose the `Student` table contains:

ID	Name	Age	Marks
1	Rahul	20	75
2	Amit	19	90
3	Priya	21	82

Sort by marks from lowest to highest:

SQL

```sql
SELECT *
FROM Student
ORDER BY Marks ASC;
```

Result:

ID	Name	Age	Marks
1	Rahul	20	75
3	Priya	21	82
2	Amit	19	90

Sort by marks from highest to lowest:

```sql
SELECT *
FROM Student
ORDER BY Marks DESC;
```

Sorting by multiple columns

You can sort using more than one column:

```sql
SELECT *
FROM Student
ORDER BY Age ASC, Marks DESC;
```

This means:

1. Sort students by **Age** from low to high.
2. If two students have the same age, sort those students by **Marks** from high to low.

Important

`ORDER BY` **does not change the actual data stored in the table.** It only changes the **order of the rows in the query result.**

you can use **`WHERE` and `ORDER BY` together** in SQL.

Syntax

SQL

```sql
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column_name ASC;
```

The important order is:

`SELECT → FROM → WHERE → ORDER BY`

Example

Suppose we have:

ID	Name	Age	Marks
1	Rahul	20	75
2	Amit	19	90
3	Priya	21	82
4	Neha	20	95

Find students whose marks are greater than 80 and sort them by marks:

```
SELECT *
FROM Student
WHERE Marks > 80
ORDER BY Marks ASC;
```

First, **WHERE filters** the records:

`90, 82, 95`

Then **ORDER BY sorts** them:

`82, 90, 95`

Result:

ID	Name	Age	Marks
3	Priya	21	82
2	Amit	19	90
4	Neha	20	95

```sql
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity
INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 4, 7);
INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2);
INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;
```

QUERY RESULTS

id	name	quantity	aisle
1	Bananas	4	7
4	Ice cream	1	12

Aggregation in SQL

Aggregation means **performing a calculation on multiple rows of data and producing a single result**.

For example, if a table contains marks of 5 students, aggregation can calculate the **total, average, highest, lowest, or number of students**.

Function	Meaning
`COUNT()`	Counts the number of records
`SUM()`	Calculates the total
`AVG()`	Calculates the average
`MAX()`	Finds the highest value
`MIN()`	Finds the lowest value

```sql
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Course VARCHAR(50),
    Age INT,
    Marks INT
);

INSERT INTO Student VALUES (1, 'Rahul', 'CSE', 20, 75);
INSERT INTO Student VALUES (2, 'Amit', 'CSE', 19, 90);
INSERT INTO Student VALUES (3, 'Priya', 'IT', 21, 82);
INSERT INTO Student VALUES (4, 'Neha', 'IT', 20, 95);
INSERT INTO Student VALUES (5, 'Raj', 'CSE', 22, 68);
INSERT INTO Student VALUES (6, 'Anjali', 'IT', 19, 88);
```

SELECT * FROM Student;

Student

StudentID	Name	Course	Age	Marks
1	Rahul	CSE	20	75
2	Amit	CSE	19	90
3	Priya	IT	21	82
4	Neha	IT	20	95
5	Raj	CSE	22	68
6	Anjali	IT	19	88

AS in SQL

AS is used to give a **temporary name (alias)** to a column or table in the query result.

```sql
SELECT AVG(Marks) AS AverageMarks
FROM Student;
```

AverageMarks
83

Average calculation

The formula for average is:

Average = Sum of all values ÷ Number of values

First calculate the sum:

75 + 90 + 82 + 95 + 68 + 88 = 498

There are **6 students**, so:

Average = 498 ÷ 6 = 83

Therefore, the average marks are **83**.

```sql
SELECT COUNT(*) AS TotalStudents
FROM Student;
```

TotalStudents
6

```sql
SELECT SUM(Marks) AS TotalMarks
FROM Student;
```

TotalMarks
498

```sql
SELECT MAX(Marks) AS HighestMarks, MIN(Marks) AS LowestMarks
FROM Student;
```

HighestMarks	LowestMarks
95	68

```sql
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 56, 7);
INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2);
INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT SUM(quantity) FROM groceries;

```

DATABASE SCHEMA

groceries	6 rows
id (PK)	INTEGER
name	TEXT
quantity	INTEGER
aisle	INTEGER

QUERY RESULTS

SUM(quantity)
67

GROUP BY in SQL

GROUP BY is used to **group rows having the same value** and then perform an **aggregate calculation on each group**.

```
SELECT Course,
AVG(Marks) AS
AverageMarks
FROM Student
GROUP BY Course;
```

Course	AverageMarks
CSE	77.66666666666667
IT	88.33333333333333

Another example: Highest and lowest marks per course

```sql
SELECT
    Course,
    MAX(Marks) AS HighestMarks,
    MIN(Marks) AS LowestMarks
FROM Student
GROUP BY Course;
```

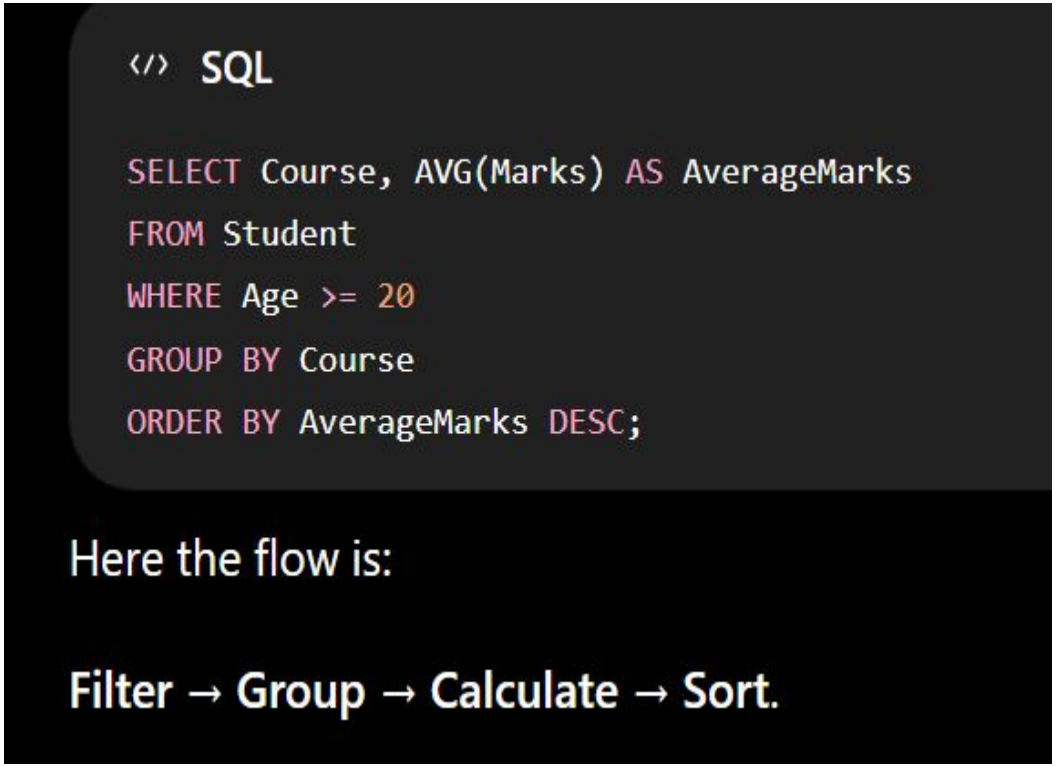
Course	HighestMarks	LowestMarks
CSE	90	68
IT	95	82

WHERE → filters rows

GROUP BY → creates groups

COUNT/SUM/AVG/MAX/MIN → calculates for each group

ORDER BY → sorts the final result



```sql
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);
INSERT INTO groceries VALUES (1, "Bananas", 56, 7);
INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);
INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);
INSERT INTO groceries VALUES(5, "Cherries", 6, 2);
INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT name, SUM(quantity) FROM groceries GROUP BY aisle;

```

DATABASE SCHEMA

groceries	6 rows
id (PK)	INTEGER
name	TEXT
quantity	INTEGER
aisle	INTEGER

QUERY RESULTS

name	SUM(quantity)
Cherries	9
Chocolate syrup	1
Bananas	56
Ice cream	1